

AALBORG UNIVERSITY

MSc in Cyber Security

Mini Project 2

TPM Attestation Experiment

Course: Security in IoT and Cloud Architectures

Group Members

Nuria Arqués León

Ikhlasse Badidi

Martha Gaki

Angelos Kamaris

Waleed Rashed

Aicha Nadja Thorman

30 October 2026

TABLE OF CONTENTS

TASK 1. SETUP TWO INDEPENDENT TPM INSTANCES 3
TASK 2. RSA SIGNING AND VERIFICATION 4
TASK 3. ECC EXTENSION 8
TASK 4. PCR-BOUND SIGNING KEY EXTENSION 9

TASK 1. SETUP TWO INDEPENDENT TPM INSTANCES

In this assignment, we used two physical TPM 2.0 chips on two separate devices. TPM-A, the signing device, is the TPM built into a Linux laptop. TPM-B, the verifying device, is a Let's Trust TPM board attached to a Raspberry Pi. The two TPMs are independent: each has its own chip, keys and PCRs, and they only communicate over the network through SSH.

Both devices

After updating, we installed TPM2-tools on both devices with sudo apt install TPM2-tools.

For **TPM-A (Laptop)**:
The laptop's TPM was already enabled in Linux. We installed the extra tools needed to run the scripts: sudo apt install python3 openssh-client.

For **TPM-B (Raspberry Pi)**:
The board contains an Infineon OPTIGA SLB 9672 chip, which works with the TPM-slb9670 driver. To enable it, we added dtoverlay=TPM-slb9670 under [all] section of /boot/firmware/config.txt:

We went into the "/boot/firmware/config.txt" and added the line: TPM-slb9670.

```
# Run in 64-bit mode
arm_64bit=1

# Disable compensation for displays with overscan
disable_overscan=1

# Run as fast as firmware / board allows
arm_boost=1

[cm4]
# Enable host mode on the 2711 built-in XHCI USB controller.
# This line should be removed if the legacy DWC2 controller is required
# (e.g. for USB device mode) or if USB support is not required.
otg_mode=1

[cm5]
dtoverlay=dwc2,dr_mode=host

[pi5]
dtoverlay=nospi10

[all]
dtoverlay=tpm-slb9670
~
~
```

We then connected the board to the Pi's GPIO pins, starting from pin 17 (3.3 V power), following the Let's Trust pinout. After powering on, the system created the TPM device interfaces /dev/TPM0 and /dev/TPMrm0. We tested the chip by asking it for 4 random bytes:

```
raspberry5@nemo:~ $ ls -l /dev/tpm*
crw-rw---- 1 tss root  10,   224 Oct 29 19:22 /dev/tpm0
crw-rw---- 1 tss tss  509, 65536 Oct 29 19:22 /dev/tpmrm0
raspberry5@nemo:~ $ sudo tpm2_getrandom 4 && echo ""
◆◆d◆
raspberry5@nemo:~ $
```

Finally, we installed an SSH server on the Pi (`sudo apt install openssh-server`), so TPM-A can send files to TPM-B and run the verification commands there.

How the scripts connect the two TPMs

run.py runs on TPM-A. When it starts, it asks for TPM-B's SSH username, address and password. TPM_utils.py handles the SSH connection, runs each TPM command on the right device, and creates the work folders and log files. Both TPMs are accessed through /dev/TPMrm0.

```
# Change these TCTI values to match each host's TPM setup.
TPM_TCTI = {"A": "device:/dev/tpmrm0", "B": "device:/dev/tpmrm0"}
SSH_TARGET = None
SSH_CONTROL = Path.home() / ".ssh" / "tpm-ex2-%C"
```

References: pinout.xyz/pinout/letstrust_TPM; ohyaan.github.io (validating a detected TPM); TPM2-tools.readthedocs.io

TASK 2. RSA SIGNING AND VERIFICATION

This task shows how TPM-A creates a TPM-backed RSA key, signs "TPM exercise", and exports only the public key and signature, while TPM-B uses them to verify the original message and detect a modified one, with evidence of both results.

TPM-A

On the TPM-A side, a primary key is first created in the owner's hierarchy, whose role is to act as the parent that protects the signing key. A child RSA-2048 signing key (RSASSA-PSS, SHA-256) is then created under it and loaded into the TPM. The key attributes (*'fixedTPM'*, *'fixedparent'*, *'sensitivedataorigin'*, *'userwithauth'* and *'sign'*), mean the private key was generated inside the TPM, cannot be duplicated or moved to another TPM, and can only be used for signing. The only private-key file ('*rsa_key.priv*') is a blob encrypted under TPM-A's primary key, so it is useless outside TPM-A.

Next, the public part of the signing key is exported to '*rsa_pub.pem*', that will be the only key file sent to TPM-B, since the private blob, the key context and the primary key will never be transferred.

```python
def create_primary_key(key_type: str, alg: dict) -> str:
    """TPM-A: create a primary key + TPM-backed signing key; return the key context path."""

    log = get_log(key_type)
    d = workdir("A")

    # Primary key (the parent type matches the child: rsa or ecc)
    log.banner(f"TPM-A: create {key_type.upper()} primary + signing key")
    flush("A")
    primary = os.path.join(d, f"{key_type}_primary.ctx")
    run(["tpm2_createprimary", "-C", "o", "-g", "sha256", "-G", key_type, "-c", primary], "A", log)

    # Signing key: RSA-2048/RSASSA-PSS or ECC P-256/ECDSA
    flush("A")
    pub, priv, ctx = os.path.join(d, f"{key_type}_key.pub"), os.path.join(d, f"{key_type}_key.priv"), os.path.join(d, f"{key_type}_key.ctx")
    # fixedtpm|fixedparent: cannot be duplicated/exported
    # sign: signing only
    run(["tpm2_create", "-C", primary, "-g", "sha256", "-G", alg["key_template"],
         "-a", "fixedtpm|fixedparent|sensitivedataorigin|userwithauth|sign",
         "-u", pub, "-r", priv], "A", log)

    # Load the key into the TPM
    flush("A")
    run(["tpm2_load", "-C", primary, "-u", pub, "-r", priv, "-c", ctx], "A", log)

    # Export only the public material required for verification
    log.banner("TPM-A: export ONLY the public key")
    pem = os.path.join(d, f"{key_type}_pub.pem")
    run(["tpm2_readpublic", "-c", ctx, "-o", pem, "-f", "pem"], "A", log)

    return ctx
```

Once `run.py` has connected to TPM-B over SSH, it creates the keys and signs the message on TPM-A, as shown below:

```
angelos-kamaris@angelos-kamaris-OMEN-Gaming-Laptop-16-am1xxx:~/Desktop/AAU-1st-Semester-Repo-Group-8/IoT/Ex2$ sudo python3 run.py
TPM-B SSH username: raspberry5
TPM-B IP address (or name): nemo.local
raspberry5@nemo.local's password:

=== TPM-A: create RSA primary + signing key ===
[TPM-A] $ tpm2_createprimary -C o -g sha256 -G rsa -c ./tpm_a/rsa_primary.ctx
name-alg:
  value: sha256
  raw: 0xb
attributes:
  value: fixedtpm|fixedparent|sensitivedataorigin|userwithauth|restricted|decrypt
  raw: 0x30072
type:
  value: rsa
  raw: 0x1
exponent: 65537
bits: 2048
scheme:
  value: null
  raw: 0x10
scheme-halg:
  value: (null)
  raw: 0x0
sym-alg:
  value: aes
  raw: 0x6
sym-mode:
  value: cfb
  raw: 0x43
sym-keybits: 128
rsa: af898d0c35c00d143113a75d69e9fca9ab941f70e83a8a12a4057cc7c08f07ec6f05453769cf3e039c75b525667ab594d65d8f038ac27032366dbf080394b42d6afc62d8fad4db455b4218e7f6d96dacddfaf1e4648734b3f49
fc701d6dfc183f336d06a3ae55e77fcb7d5a1c1dfaa4d9b217920002d9e9acb360930c4dddd340cf28b7dc1a3d50e66cfcdc8fdb7a69913089fdb721d16fae6f520535794191b53e20d46804512f40c7927911b4abd061e76849c1e6
d50a55b531827c1baf2ca132a12e2db285772e1848bf8648b07f14a8ed08039535709ced68e13c110cf2ba6d42bd335dad02dd7e1a982cf1c9ef4b5380b8510c6e9d09ab05410d273cabf
exit code: 0
```





Finally, the exact message 'TPM exercise' is signed. TPM-A hashes it with SHA-256 and signs the digest internally with the loaded RSA signing key ('*rsa_key.ctx*'). The result is stored in '*rsa_signature.sig*', and we ensure that the private key never leaves the TPM during the process.

```python
def sign_message(context: str, key_type: str, alg: dict, message="TPM exercise") -> str:
    """TPM-A: sign the message; return the message file path (on host A)."""

    log = get_log(key_type)
    d = workdir("A")
    message_file = os.path.join(d, f"{key_type}_message.txt")
    Path(message_file).write_text(message)

    log.banner(f"TPM-A: sign {message!r}")
    flush("A")
    sig = os.path.join(d, f"{key_type}_signature.sig")
    run(["tpm2_sign", "-c", context, "-g", "sha256", "-s", alg["sign_scheme"], "-o", sig, message_file], "A", log)

    return message_file
```

TPM-B

The verification starts on the TPM-B side. First, the public key, the signature, and the message file are transferred from TPM-A's folder to TPM-B via ssh and stored on the Pi in ~/.TPM_*ex2/work/TPM_*b/. TPM-B then loads the public key, which contains no secrets. It then hashes the message with SHA-256 and checks the signature against the loaded public key. If the exit code is 0, the signature is valid.

```python
def verify_signature(message_file: str, key_type: str, alg: dict, label="original message") -> bool:
    """TPM-B: verify using only the exported public key. Returns True if verified."""

    log = get_log(key_type)
    a = workdir("A")

    log.banner(f"TPM-B: verify {label}")
    # Transfer the public key, signature, and message to TPM-B
    b_pem = transfer(os.path.join(a, f"{key_type}_pub.pem"), "B", log)
    b_sig = transfer(os.path.join(a, f"{key_type}_signature.sig"), "B", log)
    b_msg = transfer(message_file, "B", log)
    ctx = os.path.join(workdir("B"), f"{key_type}_pub.ctx")

    # Load the public key into TPM-B and verify the signature
    flush("B")
    run(["tpm2_loadexternal", "-C", "n", "-G", alg["load_alg"], "-u", b_pem, "-c", ctx],"B", log)
    p = run(["tpm2_verifysignature", "-c", ctx, "-g", "sha256", "-s", b_sig, "-m", b_msg], "B", log, check=False) # a failure is expected
    flush("B")

    ok = p.returncode == 0 # 0 = verified, 1 = rejected
    log.say(f">>> {key_type.upper()} {label}: {'VERIFIED' if ok else 'REJECTED'}")
    return ok
```

The following screenshot shows TPM-B verifying the original message 'TPM exercise' and then rejecting a tampered copy:

```
=== TPM-B: verify original message ===
[transfer] rsa_pub.pem -> TPM-B host (451 bytes)
[transfer] rsa_signature.sig -> TPM-B host (262 bytes)
[transfer] rsa_message.txt -> TPM-B host (12 bytes)
[TPM-B] $ tpm2_loadexternal -C n -G rsa -u .tpm_ex2/work/tpm_b/rsa_pub.pem -c .tpm_ex2/work/tpm_b/rsa_pub.ctx
name: 000b88c99ae98cbc5962f04932989b40fab0092ec3746696d4a40c327c8817ceb1a9
exit code: 0
[TPM-B] $ tpm2_verifysignature -c .tpm_ex2/work/tpm_b/rsa_pub.ctx -g sha256 -s .tpm_ex2/work/tpm_b/rsa_signature.sig -m .tpm_ex2/work/tpm_b/rsa_message.txt
exit code: 0
>>> RSA original message: VERIFIED

TIMING RSA: key creation=2.663 s, sign=0.181 s, verify=0.732 s

=== TPM-B: verify TAMPERED message 'TPM exercise!' ===
[transfer] rsa_pub.pem -> TPM-B host (451 bytes)
[transfer] rsa_signature.sig -> TPM-B host (262 bytes)
[transfer] rsa_message_tampered.txt -> TPM-B host (13 bytes)
[TPM-B] $ tpm2_loadexternal -C n -G rsa -u .tpm_ex2/work/tpm_b/rsa_pub.pem -c .tpm_ex2/work/tpm_b/rsa_pub.ctx
name: 000b88c99ae98cbc5962f04932989b40fab0092ec3746696d4a40c327c8817ceb1a9
exit code: 0
[TPM-B] $ tpm2_verifysignature -c .tpm_ex2/work/tpm_b/rsa_pub.ctx -g sha256 -s .tpm_ex2/work/tpm_b/rsa_signature.sig -m .tpm_ex2/work/tpm_b/rsa_message_tampered.txt
WARNING:esys:src/tss2-esys/api/Esys_VerifySignature.c:302:Esys_VerifySignature_Finish() Received TPM Error
ERROR:esys:src/tss2-esys/api/Esys_VerifySignature.c:103:Esys_VerifySignature() Esys Finish ErrorCode (0x000002db)
ERROR: Esys_VerifySignature(0x2DB) - tpm:parameter(2):the signature is not valid
ERROR: Verify signature failed!
ERROR: Unable to run tpm2_verifysignature
exit code: 1
>>> RSA TAMPERED message 'TPM exercise!': REJECTED

RSA RESULT: original verified=True, tampered rejected=True -> PASS
```

This valid result shows that the signature was produced by the holder of the private key that matches '*rsa_pub.pem*', over exactly this message.

We also demonstrate that verification fails if the message is modified. We create a new text file containing 'TPM exercise!' and check it against the same signature and the same public key. Because the message changes, its digest changes, the check fails, and the command exits with a non-zero code.

The rejection of this tampered message shows that changing even one character invalidates the signature, so TPM-B detects any modification of the data. The error code '*0x2DB'* corresponds to '*TPM_RC_SIGNATURE'*, and '*parameter(2)'* indicates that the second input, the signature, does not match the supplied data.

Finally, all logs are in the evidence folder under the name 'rsa_sign_verify.log'. It contains each step performed during the task, which is the same as it was written in the command line.

TASK 3. ECC EXTENSION

For this task, TPM-A uses an ECC signing key instead of RSA, but the workflow is the same as in Task 2.

The command sequence, key attributes, hash, and message are identical, and only these parameters changed within the implementation:

	RSA	**ECC**
Primary key	-G rsa	-G ecc
Key creation	-G rsa2048:rsapss-sha256:null	-G ecc256:ecdsa-sha256
Signing scheme	-s rsapss	-s ecdsa
Loading the pub key on TPM-B	-G rsa	-G ecc

The main differences are the sizes. The RSA signature file is 262 bytes (256 bytes of signature plus a 6-byte TSS header, which are small labels that say how to read the signature), while the ECC signature file is 72 bytes (64 bytes of signature plus an 8-byte TSS header). The public key file '*rsa_pub.pem*' is 451 bytes, against 178 bytes for '*ecc_pub.pem*'. ECC therefore gives smaller keys and signatures for comparable security (about 128 bits for P-256 against about 112 bits for RSA-2048). Further, ECC was faster than RSA in every step (key creation 0.452 s vs 2.663 s, signing 0.161 s vs 0.181 s, and verification 0.464 s vs 0.732 s), most clearly in key creation, where RSA must generate large prime numbers. These times are slower than in our earlier simulator tests, which is expected on real hardware.

However, the rest remains the same. TPM-A creates a primary key (type ECC) in the owner hierarchy, then creates and loads a child ECC signing key (NIST P-256, ECDSA, SHA-256) under it. Because the key attributes are unchanged, the private key stays inside the TPM. The public key is exported to '*ecc_pub.pem*', and the same message 'TPM exercise' is signed inside the TPM, producing '*ecc_signature.sig*'. TPM-B receives the public key, signature and message, loads the public key and verifies the signature. The screenshot shows TPM-B verifying the original message and then rejecting the tampered copy:

```
=== TPM-B: verify original message ===
[transfer] ecc_pub.pem -> TPM-B host (178 bytes)
[transfer] ecc_signature.sig -> TPM-B host (72 bytes)
[transfer] ecc_message.txt -> TPM-B host (12 bytes)
[TPM-B] $ tpm2_loadexternal -C n -G ecc -u .tpm_ex2/work/tpm_b/ecc_pub.pem -c .tpm_ex2/work/tpm_b/ecc_pub.ctx
name: 000bbe5b6d99034811c334138238fa81655db488d73b466a274fb4060ff41a788e63
exit code: 0
[TPM-B] $ tpm2_verifysignature -c .tpm_ex2/work/tpm_b/ecc_pub.ctx -g sha256 -s .tpm_ex2/work/tpm_b/ecc_signature.sig -m .tpm_ex2/work/tpm_b/ecc_message.txt
exit code: 0
>>> ECC original message: VERIFIED

TIMING ECC: key creation=0.452 s, sign=0.161 s, verify=0.464 s

=== TPM-B: verify TAMPERED message 'TPM exercise!' ===
[transfer] ecc_pub.pem -> TPM-B host (178 bytes)
[transfer] ecc_signature.sig -> TPM-B host (72 bytes)
[transfer] ecc_message_tampered.txt -> TPM-B host (13 bytes)
[TPM-B] $ tpm2_loadexternal -C n -G ecc -u .tpm_ex2/work/tpm_b/ecc_pub.pem -c .tpm_ex2/work/tpm_b/ecc_pub.ctx
name: 000bbe5b6d99034811c334138238fa81655db488d73b466a274fb4060ff41a788e63
exit code: 0
[TPM-B] $ tpm2_verifysignature -c .tpm_ex2/work/tpm_b/ecc_pub.ctx -g sha256 -s .tpm_ex2/work/tpm_b/ecc_signature.sig -m .tpm_ex2/work/tpm_b/ecc_message_tampered.txt
WARNING:esys:src/tss2-esys/api/Esys_VerifySignature.c:302:Esys_VerifySignature_Finish() Received TPM Error
ERROR:esys:src/tss2-esys/api/Esys_VerifySignature.c:103:Esys_VerifySignature() Esys Finish ErrorCode (0x000002db)
ERROR: Esys_VerifySignature(0x2DB) - tpm:parameter(2):the signature is not valid
ERROR: Verify signature failed!
ERROR: Unable to run tpm2_verifysignature
exit code: 1
>>> ECC TAMPERED message 'TPM exercise!': REJECTED

ECC RESULT: original verified=True, tampered rejected=True -> PASS
```

When we check the same signature against the tampered message 'TPM exercise!', the verification fails with error *'0x2DB'* ('*TPM_RC_SIGNATURE*') and exits with code 1, just like RSA.

The logs for this task are saved in '*ecc_sign_verify.log*'.

TASK 4. PCR-BOUND SIGNING KEY EXTENSION

In this task, we extended the experiment in a way that the signing key on TPM-A can only be used when selected PCR registers match a predefined expected value. Then, we demonstrated the following:

1. TPM A's signing key is restricted in such task so it can be used only when PCR 23 has a predefined expected value. TPM-A is on a Linux laptop and TPM-B is on the Raspberry Pi.

```
=== 1. TPM-A: set the predefined (expected) PCR state ===
[TPM-A] $ tpm2_pcrreset 23
exit code: 0
[TPM-A] $ tpm2_pcrevent 23 ./tpm_a/measurement.txt
sha1: a3311c379391c8bd3027f95b6eebeb2836514ca7
sha256: e74dae677f8de0ed535c3c7a67a0ca033f8b8c97dff03abc48acfd14c71fb411
sha384: c46085d80a2650bfe792bebc3226e350054a7758708c668db347c8fe120a5bd623b03ae846b08e67d3f881b3b22dfcb0
sm3_256: 4cd70c1beacbfae8ec12bbc7b70b2ea57089578eb1ebd2aee07bb4073d44c657
exit code: 0
[TPM-A] $ tpm2_pcrread sha256:23
sha256:
    23: 0xAE70DB96DC9B971D31BF4250D083828218FD828A3FA962FBCA54790A674E40CB
exit code: 0
```

2. a.

```
=== 2a. TPM-A: create the PCR policy ===
[TPM-A] $ tpm2_startauthsession -S ./tpm_a/trial.ctx
exit code: 0
[TPM-A] $ tpm2_policypcr -S ./tpm_a/trial.ctx -l sha256:23 -L ./tpm_a/pcr.policy
ace9970895d2927083cbb33a4cbf526b6d215151b7ae75dd0e18bcda6437f354
exit code: 0
[TPM-A] $ tpm2_flushcontext ./tpm_a/trial.ctx
exit code: 0
policy digest: ace9970895d2927083cbb33a4cbf526b6d215151b7ae75dd0e18bcda6437f354
```

Here, we created a rule which basically says "PCR 23 must have this expected value, ignore anything else". The TPM stores this rule as a short fingerprint called Policy Digest. `ace9970895d2927083cbb33a4cbf526b6d215151b7ae75dd0e18bcda6437f354`

```
=== 2b. TPM-A: create a signing key restricted by the PCR policy ===
[TPM-A] $ tpm2_createprimary -C o -g sha256 -G ecc -c ./tpm_a/pcr_primary.ctx
name-alg:
  value: sha256
  raw: 0xb
attributes:
  value: fixedtpm|fixedparent|sensitivedataorigin|userwithauth|restricted|decrypt
  raw: 0x30072
type:
  value: ecc
  raw: 0x23
curve-id:
  value: NIST p256
  raw: 0x3
kdfa-alg:
  value: null
  raw: 0x10
kdfa-halg:
  value: (null)
  raw: 0x0
scheme:
  value: null
  raw: 0x10
scheme-halg:
  value: (null)
  raw: 0x0
sym-alg:
  value: aes
  raw: 0x6
sym-mode:
  value: cfb
  raw: 0x43
sym-keybits: 128
x: e33acd36ea57cf1b9682c61250a6ea0b62ad1ae30b62bf7a287e6f4edba52489
y: b60a7add80ce6206574cf6844b744ff26e4fa7d301514a19af0a62877bac281a
exit code: 0
```

Just like Tasks 2 and 3, we create a primary key in the owner hierarchy in TPM-A. It will act as the parent that protects the signing key.

```
=== 2c. TPM-A: export ONLY the public key ===
[TPM-A] $ tpm2_readpublic -c ./tpm_a/pcr_key.ctx -o ./tpm_a/pcr_pub.pem -f pem
name: 000b028ea9254c72307dc5a1b4801f10119f24e9d9f7f745b91d9cb0a76154392b96
qualified name: 000b5a9f46a772c88403b06986e6f25f2c3c51565240c942739fc548290f3b953eab
name-alg:
  value: sha256
  raw: 0xb
attributes:
  value: fixedtpm|fixedparent|sensitivedataorigin|sign
  raw: 0x40032
type:
  value: ecc
  raw: 0x23
curve-id:
  value: NIST p256
  raw: 0x3
kdfa-alg:
  value: null
  raw: 0x10
kdfa-halg:
  value: (null)
  raw: 0x0
scheme:
  value: ecdsa
  raw: 0x18
scheme-halg:
  value: sha256
  raw: 0xb
sym-alg:
  value: null
  raw: 0x10
sym-mode:
  value: (null)
  raw: 0x0
sym-keybits: 0
x: e260104ad2f9d1d0cddc8fab6ddcf7d494160a05fea2adf13bbb494362bc0796
y: 4b1008d3076ff8c7fa36c1bc3938ead58e41c5ab65a457ff426893223b40bd3b
authorization policy: ace9970895d2927083cbb33a4cbf526b6d215151b7ae75dd0e18bcda6437f354
exit code: 0
```

We then created an ECC signing key that has the policy digest attached. We removed the "use with password" option, so we force the TPM to use the PRC rule to use the key. The screenshots confirms that "userwithauth" is missing from its attributes and the key's auth. Policy matches the rule.

3.

```
=== TPM-A: sign 'TPM exercise' (PCR matches) ===
[TPM-A] $ tpm2_startauthsession --policy-session -S ./tpm_a/session.ctx
exit code: 0
[TPM-A] $ tpm2_policypcr -S ./tpm_a/session.ctx -l sha256:23
ace9970895d2927083cbb33a4cbf526b6d215151b7ae75dd0e18bcda6437f354
exit code: 0
[TPM-A] $ tpm2_sign -c ./tpm_a/pcr_key.ctx -g sha256 -s ecdsa -p session:./tpm_a/session.ctx -o ./tpm_a/pcr_signature.sig ./tpm_a/pcr_message.txt
exit code: 0
>>> Signing (PCR matches): ALLOWED
```

Since the PCR 23 had the expected value and the session produced the same fingerprint as the key policy, the TPM allowed the key to sign "TPM exercise." We can confirm that by seeing exit code: 0 >>> Signing: ALLOWED.

4.

```
=== TPM-A: change PCR 23 (untrusted software measured) ===
[TPM-A] $ tpm2_pcrevent 23 ./tpm_a/untrusted.txt
sha1: 9cc65d316894cf4ec17aa4fc83db72f2d01104f3
sha256: 37a8ef2fe4e0b5f1ad40173330bd34494e841d9c46661354e56325d78f3fcf7c
sha384: 0b0bf0e787d229ecf7329efadb52f7dda996af3d06327b423fe33eccab90829e4571fbc0d22c310711d60b733dedfb7f
sm3_256: 41461b15cfc8c74a992cde7a711027b783f55292a5c8626923e3bf76a8ab85a5
exit code: 0
[TPM-A] $ tpm2_pcrread sha256:23
sha256:
    23: 0xBF34CF5B1E732A23043C369D0216E5D791397746C928073EC248240AA2CF8011
exit code: 0
```

We now recorded an untrusted file into the PCR, `(untrusted software measured)`, so its value no longer matches the expected state.

```
=== TPM-A: sign 'TPM exercise' (PCR changed) ===
[TPM-A] $ tpm2_startauthsession --policy-session -S ./tpm_a/session.ctx
exit code: 0
[TPM-A] $ tpm2_policypcr -S ./tpm_a/session.ctx -l sha256:23
64935ae6989d208903577378fe9b8b9c702cbc1bdf9acfd40945fdc1f7817e20
exit code: 0
[TPM-A] $ tpm2_sign -c ./tpm_a/pcr_key.ctx -g sha256 -s ecdsa -p session:./tpm_a/session.ctx -o ./tpm_a/pcr_signature_bad.sig ./tpm_a/pcr_message.txt
WARNING:esys:src/tss2-esys/api/Esys_Sign.c:311:Esys_Sign_Finish() Received TPM Error
ERROR:esys:src/tss2-esys/api/Esys_Sign.c:105:Esys_Sign() Esys Finish ErrorCode (0x0000099d)
ERROR: Esys_Sign(0x99D) - tpm:session(1):a policy check failed
ERROR: Unable to run tpm2_sign
exit code: 1
>>> Signing (PCR changed): REFUSED
```

We then ran the exact same steps as before but this session produced a different fingerprint from the key's policy. (64935ae6... ≠ ace99708...). The TPM saw the mismatch in fingerprints and refused to sign and gave us the error: `ERROR: Esys_Sign(0x99D) - tpm:session(1):a policy check failed` `ERROR: Unable to run tpm2_sign`. Now, exit code: 1 >>> Signing: REFUSED.

5.

```
=== 5. TPM-B: verify the signature made under the valid PCR state ===
[transfer] pcr_pub.pem -> TPM-B host (178 bytes)
[transfer] pcr_signature.sig -> TPM-B host (72 bytes)
[transfer] pcr_message.txt -> TPM-B host (12 bytes)
[TPM-B] $ tpm2_loadexternal -C n -G ecc -u .tpm_ex2/work/tpm_b/pcr_pub.pem -c .tpm_ex2/work/tpm_b/pcr_pub.ctx
name: 000bfce34c67c408672a9e856aec0b7d0625e61a1b36d9b037bf1b11813b5a78c878
exit code: 0
[TPM-B] $ tpm2_verifysignature -c .tpm_ex2/work/tpm_b/pcr_pub.ctx -g sha256 -s .tpm_ex2/work/tpm_b/pcr_signature.sig -m .tpm_ex2/work/tpm_b/pcr_message.txt
exit code: 0
>>> TPM-B verification: VERIFIED

PCR RESULT: signing allowed with expected PCR=True, signing refused after PCR change=True, TPM-B verified=True -> PASS
Exit request sent.
```

We sent the message, the public key, and the signature (from step 3) to TPM-B on the Raspberry PI over SSH. TPM-B ended up verifying the signature successfully.

```
PCR RESULT: signing allowed with expected PCR=True, signing refused after PCR change=True, TPM-B verified=True -> PASS
Exit request sent.
```

This last line confirms that all three checks passed.

After run.py has run successfully, the files mentioned before will appear in the dir running run.py and the directory .TPM_ex2 in the system with TPM-b.



The above folder listings show what each TPM holds. TPM-A's folder on the laptop contains the private key (*.priv), while TPM-B's folder on the Pi only contains public keys, messages, and signatures. The private key never left TPM-A. The full output could be found saved in evidence/pcr_bound_signing.log.

Explain what this proves and what it does not prove

What it proves:

- The TPM enforces the rules, not the software. The same commands were run twice, having only the PCR 23 change, and the TPM refused the second time.
- The private key stays inside TPM-A and can only sign when the PCR state matches.
- The key is protected by a real hardware TPM, and the signature was verified by another physically separate TPM over a real network connection.

What it doesn't prove:

- TPM-B does not see the PCR policy since nothing about the PCR is included in the signature, so TPM-B only confirms that the public key signed the message. It must assume the key is bound to a PCR policy.
- “Trusted” only means the value matches what was recorded. PCR says nothing about whether the device is actually safe.
- PCR 23 can be reset by anyone who has access to the device and they can replay the trusted measurement to unlock the key again.